Monotonicity as an Architectural Bias for Robust Language Models

Anonymous authors
Paper under double-blind review

Abstract

Large language models (LLMs) are known to exhibit brittle behavior under adversarial prompts and jailbreak attacks, even after extensive alignment and fine-tuning: small, carefully structured perturbations in high-dimensional input spaces can induce large, unpredictable changes in internal semantic representations and output.

We investigate monotonicity—constraining semantic transformations so strengthening information, evidence, or constraints cannot regress internal representations—as an architectural inductive bias for Transformer robustness, long exploited in control and safety-critical systems but traditionally viewed as incompatible with the expressivity neural language models require.

We show this trade-off is not inherent: enforcing monotonicity selectively in the feed-forward sublayers of sequence-to-sequence Transformers—leaving attention unconstrained—yields monotone language models that preserve pretrained performance, letting negation, contradiction, and contextual interactions enter explicitly through attention while subsequent refinement stays order-preserving. Empirically, monotonicity substantially improves robustness: adversarial attack success rates drop from approximately 69% to 19%, with only marginal summarization degradation. This benefit transfers to a decoder-only foundation model (Pythia-1.4B), but only when constraint scope matches the architecture: the feed-forward input projection alone—sufficient in the encoder-decoder setting—fails outright, whereas constraining both feed-forward projections reduces the mean attack success rate from 69.0% to 42.5% across three seeds, at a larger perplexity cost. Monotonicity is thus a practical inductive bias for robustness, with an architecture-dependent quality–robustness trade-off.

1 Introduction

Large language models (LLMs) achieve remarkable capabilities across natural language tasks, yet their behavior under adversarial or structured inputs remains brittle: even extensively aligned models can be induced to produce unsafe outputs through jailbreak prompts or small, targeted perturbations. This reflects a deeper issue—modern LLMs operate in extremely high-dimensional spaces where subtle changes can induce large, unpredictable output shifts, a sensitivity recently attributed to the high Lipschitz constants such models exhibit, amplifying perturbations as they propagate through the network Newhouse et al. (2025).

This motivates a growing line of robustness research Salah (2026); Su et al. (2024)—training-time (alignment Cao et al. (2024), adversarial training Howe et al. (2024)) or inference-time (output filtering Khachaturov & Mullins (2025))—but inherently reactive, raising the question of whether models can instead be given structural properties that make behavior predictable under perturbation. We investigate monotonicity as such a bias: constraining outputs to change predictably with the input, with a long history in machine learning Gupta et al. (2016); Runje & Shankaranarayana (2023) and related fields Smith (1995), especially influential in control theory, where monotone systems admit strong stability guarantees in

high-dimensional settings Michel et al. (2015); Ito et al. (2014)—e.g., certifying power grids, where reasoning over all system states is infeasible Rantzer & Bernhardsson (2014).

Monotonicity also matches how we expect language models to behave semantically—adding evidence should not weaken a conclusion, adding safety constraints should not loosen internal representations—yet contemporary LLMs frequently violate these expectations. Concretely, prompts that progressively strengthen a factual claim ("The medication has side effects" $\preceq$ "The medication has severe side effects" $\preceq$ "...and requires medical supervision") form a partial order that a model's representations and outputs should respect. This reflects a structural source of brittleness: attention can introduce operators like negation explicitly by linking tokens, but unconstrained feed-forward transformations may arbitrarily amplify, suppress, or cancel features, so semantic reversals occur implicitly through distributed feature cancellation—a key contributor, we argue, to brittle behavior under adversarial prompts, and a close parallel to monotone Boolean circuits Jukna et al. (2012), where negation is disallowed within the circuit and must be introduced explicitly at the inputs, preserving expressive power while enforcing semantic discipline. Analogously, enforcing monotonicity encourages language models to represent negation or constraint relaxation explicitly through attention rather than implicitly through fragile nonlinear cancellations—separating where semantic reversals may occur from where meaning is refined, and, as in monotone dynamical systems, simplifying reasoning about global behavior since it suffices to verify satisfaction only at extreme points Feyzmahdavian et al. (2017).

Our central finding is that monotonicity can be incorporated into Transformer architectures without sacrificing performance, by enforcing it selectively within feed-forward sublayers while leaving attention unconstrained: monotone Transformers match non-monotone task performance and show substantially improved adversarial and jailbreak robustness, gains arising purely from architectural structure. This challenges the prevailing view Sartor et al. (2025) that strong architectural constraints inevitably degrade optimization or expressive capacity; instead, monotonicity can serve as a practical, principled inductive bias, developed here through formal analysis and empirical evaluation.

Contributions. Our contributions are as follows:

1. Monotone Transformer components. We introduce monotone feed-forward sublayers for Transformers, enforcing structural monotonicity while leaving attention unconstrained.
2. Lossless distillation. Pretrained Transformers can be distilled into monotone counterparts without degrading standard benchmark performance.
3. Improved robustness. Monotone language models empirically exhibit increased robustness to adversarial and jailbreak attacks.
4. Architecture-conditional transfer. We characterize how the robustness benefit transfers from encoder-decoder to decoder-only architectures: decoder-only models require broader constraint coverage to close adversarial leak paths, at a quantified quality–robustness trade-off on Pythia-1.4B.

2 Related Work

Vulnerabilities and Defenses. LLMs are highly susceptible to adversarial manipulations bypassing alignment: transferable adversarial suffixes (Zou et al., 2023), position-sensitive jailbreaks (Wang et al., 2025b), perplexity-evading attacks (Zhu et al., 2023), and context-scaling many-shot jailbreaks (Anil et al., 2024). Robustness benchmarks show poor performance under systematic evaluation (Wang et al., 2021) and multilingual safety failures (Deng et al., 2024), while prompt injection is a fundamental vulnerability alongside broader threats (Li & Fung, 2025). Proposed defenses include provable smoothing-based guarantees (Robey et al., 2023), lightweight output-distribution detection (Sayeedi et al., 2025), scale-dependent adversarial training (Howe et al., 2024), and attack-transferability studies (Wang et al., 2025a)—but most remain reactive and training-dependent.

Monotonic Neural Networks and Structural Constraints. Monotonicity has long served as a structural bias for interpretability and robustness: non-negative-weight, monotone-activation networks are universal approximators of monotone functions (Sill, 1998; Daniels

& Velikova, 2010). Subsequent work proposed practical monotone architectures, including lattice-based models (Gupta et al., 2016; You et al., 2017), and verification benefits from boundary-input reasoning (Weber et al., 2021); structured layers more broadly improve robustness without prohibitive loss (Amos & Kolter, 2017). We build on this by scaling selectively enforced monotonicity to Transformer-based LLMs with tangible gains.

3 Monotone Transformers

We study monotonicity constraints for improving the adversarial robustness of sequence-to-sequence (Seq2Seq) language models.

Definition 3.1 (Sequence-to-Sequence Model). A sequence-to-sequence model is a parameterized mapping $\mathcal{M}_\theta : \mathcal{X} \to \mathcal{Y}$, where $\mathcal{X}, \mathcal{Y}$ are the input/output token-sequence spaces. Here $\mathcal{M}_\theta$ is a Transformer composed of attention layers, feed-forward network (FFN) sublayers, residual connections, and normalization.

We formalize monotonicity on hidden-state coordinates via the coordinatewise (product) order $h \leq h'$ on $\mathbb{R}^d$, interpreting $h' \geq h$ as strengthening every coordinate—the order our FFN sublayers (below) respect.

Section 4.3 separately asks whether this induces order preservation along interpretable semantic directions $A \in \mathbb{R}^{p \times d}$, recovered post hoc via probes on ordered prompt pairs (Section 1)—a diagnostic measurement only, since A is fit after training and plays no role in enforcing monotonicity below.

Definition 3.2 (Monotonicity). A function $f : \mathbb{R}^n \to \mathbb{R}^m$ is monotone if for all $x, x' \in \mathbb{R}^n$, $x \leq x' \;\Rightarrow\; f(x) \leq f(x')$.

Lemma 3.3 (Closure under Composition). If $f : \mathbb{R}^n \to \mathbb{R}^m$ and $g : \mathbb{R}^m \to \mathbb{R}^k$ are monotone, then their composition $g \circ f$ is monotone.

Consider a feed-forward neural network $N : \mathbb{R}^{n_0} \to \mathbb{R}^{n_k}$ with k hidden layers. For input $u \in \mathbb{R}^{n_0}$,

$$\begin{aligned} y_0 &= u, \\ y_{i+1} &= \sigma(W_i y_i + b_i), \quad i = 0, \ldots, k-1, \\ N(u) &= W_k y_k + b_k, \end{aligned}$$

where σ is applied elementwise.

Proposition 3.4 (Sufficient Condition for FFN Monotonicity). If σ is elementwise non-decreasing and all weight matrices satisfy $W_i \geq 0$ for $i = 0, \ldots, k$, then N is monotone.

Proof. Each affine map with nonnegative weights is monotone, and σ preserves monotonicity by assumption. The claim follows by Lemma 3.3. □

Networks satisfying Proposition 3.4 are called monotone neural networks, and are universal approximators of continuous monotone functions on compact domains Daniels & Velikova (2010).

Definition 3.5 (Monotone Transformer). A Transformer model $\mathcal{M}_\theta$ is monotone with respect to its feed-forward sublayers if each FFN sublayer implements a residual update $F(h) := h + N(h)$, $N(h) = W_2\,\sigma(W_1 h + b_1) + b_2$, where W_1, W_2 satisfy Proposition 3.4 (elementwise nonnegative, σ non-decreasing), so N is monotone on $\mathbb{R}^d$ under the coordinatewise order; all other components (attention, residuals, normalization) remain unconstrained.

Since the identity map is trivially monotone and sums of monotone maps are monotone, $F = \mathrm{id} + N$ is itself monotone whenever N is, so residuals need no separate constraint; this enforces monotonicity coordinatewise at the FFN sublayers—not globally—where implicit feature cancellation is most likely Runje & Shankaranarayana (2023); Sartor et al. (2025); Nguyen et al. (2023).

We enforce Proposition 3.4's nonnegativity by constraining every FFN weight matrix (W_1, W_2) elementwise nonnegative, retaining the pretrained activation. Each constrained

$W \in \mathbb{R}^{m \times n}$ is reparameterized $W = \phi(V)$, $\phi(v) = \log(1+\exp(v))$, with unconstrained V (applied elementwise), guaranteeing $W \geq 0$ by construction, without projection or constrained optimization.

Naively initializing V so $W \approx 0$ would discard the pretrained weights entirely; instead we set $\phi(V_{\text{init}}) = |W_{\text{pre}}| + \epsilon$ ($\epsilon > 0$ small), preserving the pretrained scale while flipping every negative weight's sign to satisfy $W \geq 0$ (biases unchanged). Since roughly half of a typical pretrained weight matrix is negative, this substantially changes each FFN sublayer's function—consistent with the elevated initial training loss in Table 1—while preserving more information than a small random initialization.

4 Experimental Setup

We instantiate the monotone Transformer framework of Section 3 using T5 Raffel et al. (2020), applying Definition 3.5 to every FFN sublayer by reparameterizing both weight matrices (W_1, W_2) with the softplus parameterization of Section 3, retaining the pretrained activation; attention, residuals, and normalization remain unconstrained.

- Model. T5-small: 6 encoder + 6 decoder layers, 512-dim hidden state, 8 heads, 2048-dim FFN; ≈60M parameters (≈24M, 40%, in FFN weights), all constrained elementwise nonnegative—monotonicity is enforced coordinatewise on the full 512-dim hidden state at every FFN sublayer, with no bottleneck.
- Optimization. AdamW, lr 5×10^{-5}, weight decay 0.01, batch size 4, gradient clipping norm 1.0, 7 epochs; monotone model uses extended 15% warmup (vs. 10% baseline).
- Data. Trained on DialogSum, HighlightSum, and arXiv abstracts (≈150K examples); evaluated on held-out CNN/DailyMail, XSUM, and SAMSum (CNN/DailyMail excluded from training to assess out-of-distribution generalization).
- Decoding. Beam search (4 beams), length penalty 1.2, min/max length 10/80 tokens, no-repeat 3-gram blocking.

Full optimization, data, decoding, and hardware details, and the multi-seed/full-test-set validation protocol, are given in Appendix A.2.

4.1 Adversarial Robustness Evaluation

We evaluate two attack families: UATs Wallace et al. (2019), input-agnostic token sequences optimized via coordinate-wise search (three restarts, 50 iterations) to maximize loss when prepended to any input, with a transfer matrix measuring cross-model vulnerability; and HotFlip Ebrahimi et al. (2018), which replaces up to five gradient-selected tokens per example. We report ROUGE degradation, attack success rate (ROUGE-L degradation $> 10\%$), mean loss increase, and trigger transferability, with significance via paired t-tests (Bonferroni-corrected) and Cohen's d; Appendix A.2 gives full attack hyperparameters.

4.2 HotFlip Substitution Transfer and Gradient-Free Controls

To test whether HotFlip success-rate gaps are genuine or an artifact of each attack exploiting its own loss landscape, we replay each source model's substitutions as fixed inputs against every target, yielding a 2×2 transfer matrix, alongside gradient-free random-substitution and query-based controls ruling out gradient masking. Full protocol in Appendix A.5.

4.3 End-to-End Order Preservation

Definition 3.5 enforces coordinatewise monotonicity per FFN sublayer, not along semantic directions. We test whether this induces end-to-end order preservation via linear probes on a corpus of ordered prompt pairs strengthening along thirteen semantic axes over eighteen topics (234 chains, 702 pairs), measuring per layer the fraction of probe coordinates satisfying $Ah \leq Ah'$ on a held-out split. Corpus, probe, and pooling details are in Appendix A.5.

4.4 Attribution Ablations

The softplus reparameterization of Section 3 both imposes elementwise nonnegativity and disrupts the pretrained weight geometry. Two control arms isolate these factors: a sign-frozen control reconstructing the pretrained mixed-sign pattern (not monotone), and an init-disruption control that is unconstrained but shares the monotone arm's initialization. If robustness vanishes under sign-frozen, nonnegativity is causal; if it persists, reparameterization/initialization alone matters. Full protocol in Appendix A.5.

4.5 Decoder-Only Foundation Model Track

To test whether this relationship generalizes to decoder-only architectures, we conduct a second track using Pythia-1.4B Biderman et al. (2023), a 24-layer autoregressive GPT-NeoX model fine-tuned and evaluated on the Pile Gao et al. (2020) (uncopyrighted subset), with clean performance measured by perplexity—exponentiated mean NLL per token—on a held-out sample.

Because decoder-only blocks lack T5's encoder-decoder bottleneck, which sublayers must be constrained is not obvious. Each GPT-NeoX block has an attention output projection and an MLP with input ($W_{\text{in}} \in \mathbb{R}^{4d\times d}$) and output ($W_{\text{out}} \in \mathbb{R}^{d\times 4d}$) projections. We compare three constraint scopes (softplus reparameterization of Section 3, applied in all 24 layers):

- MLP-in: constrain only W_{in}, the direct analogue of the T5 scheme;
- MLP-both: constrain both W_{in} and W_{out};
- MLP-in + Attn-out: constrain W_{in} and the attention output projection.

After imposing constraints, each model undergoes recovery training on the Pile, with the baseline fine-tuned identically; scopes are screened on one seed under a reduced budget, then validated under the full budget across three seeds. Robustness uses the same attack implementations, transfer-matrix, and gradient-free controls (Section 4.2) as the T5 track, with the ordered-pair probe protocol of Section 4.3 applied to decoder residual streams. Full details in Appendix A.4.

4.6 Scale-Up Protocol

The main-text tables report the already-complete T5-small and Pythia-1.4B arms. To test whether the quality–robustness pattern persists at larger capacity, the same pipelines accept a model-name override with size-tier presets: the T5 track on t5-base/t5-large, the decoder-only track on Pythia-2.8B/6.9B, and a second decoder-only route on Llama 3 models Grattafiori et al. (2024), whose SwiGLU feed-forward sublayer falls outside Proposition 3.4 and is treated as a stress test of the nonnegativity constraint. Appendix A.5 gives full size tiers, seed counts, constraint variants, and hardware; results replace the modest-scale limitation once the corresponding JSON artifacts exist.

5 Mechanisms of Gradient Attenuation in Monotone Feed-Forward Networks

We give a mechanistic (not global robustness) explanation for these gains: monotonicity alters the local gradient structure of FFN sublayers, reducing the effectiveness of gradient-based attacks such as HotFlip, motivated by monotone dynamical systems where derivatives along strongly monotone flows tend to zero near an invariant set Hirsch (1985) (Appendix A.1 gives full derivations below). Consider a monotone FFN update $F(h) := h+N(h)$, $N(h) = W_2\sigma(W_1h + b_1) + b_2$, as in Definition 3.5.

Lemma 5.1 (Non-negativity of the FFN Jacobian). If $N(h) = W_2\sigma(W_1h + b_1) + b_2$ with $W_1, W_2 \geq 0$ elementwise and σ elementwise non-decreasing, then the Jacobian $J_F(h) = \nabla_h F(h) = I + J_N(h)$ has non-negative entries for all $h \in \mathbb{R}^d$.

Intuitively, increasing any hidden coordinate cannot decrease any output coordinate of F, ruling out sign cancellation inside the FFN sublayer; since backpropagated gradients are

filtered by $J_F(h)^\top$, this bounds how the sublayer can reshape gradient signal. Proof in Appendix A.1.

5.1 Saturation-Induced Gradient Attenuation

Lemma 5.1 holds for any non-decreasing σ, including our T5-small's ReLU; we additionally analyze a complementary effect for saturating activations.

Assumption 5.2 (Saturating Activations). The activation σ has bounded derivative and admits saturated regimes, i.e., $\sigma'(z)$ becomes arbitrarily small as $z \to +\infty$.

Lemma 5.3 (Gradient Attenuation under Saturation). Let $F(h) = h+N(h)$ with σ satisfying Assumption 5.2. If, along a sequence $\{h_k\}$, a non-empty subset of pre-activations in N diverges to $+\infty$, the contribution of the corresponding saturated units to $J_N(h_k)$—and hence to $\nabla_h \mathcal{L}(F(h_k))$—vanishes asymptotically.

This shows monotonicity, combined with saturation, can locally attenuate gradient magnitude without eliminating all signal. Proof in Appendix A.1.

5.2 Persistence of Saturation Effects

Lemma 5.4 (Persistence of Saturated Units). Let $h \le h'$ and $z_j(h) = (W_1 h + b_1)_j$, with $W_1 \ge 0$ elementwise. If unit j is saturated at h', it remains saturated at $h' + \delta$ for any monotone perturbation $\delta \ge 0$.

This is a one-sided stability property of individual units, not the entire input gradient. Proof in Appendix A.1. Our T5-small experiments use ReLU, for which Lemma 5.1 applies unconditionally but Lemma 5.4 does not transfer directly, since ReLU saturates only on its inactive side rather than its active side; Appendix A.1 discusses this distinction in full.

5.3 Implications for Gradient-Based Attacks

Gradient-based attacks such as HotFlip rely on persistent, high-magnitude gradients with respect to token embeddings. Monotonicity reduces their availability both directly (Lemma 5.1) and, for bounded activations, via saturated units that do not recover under monotone increases (Lemmas 5.3–5.4)—intuition rather than a formal guarantee, since monotonicity does not prevent all attacks or eliminate gradients globally. Section 6.3 probes this account by testing whether the robustness gain survives gradient-free and cross-model attack transfer.

6 Results

We evaluate monotonicity's impact on task performance and adversarial robustness: optimization and summarization quality on the T5 track, then robustness under targeted attacks, concluding with the decoder-only track (Section 6.4) testing whether the benefit transfers across architectures.

6.1 Training Dynamics and Summarization Quality

Table 1 summarizes training dynamics: the baseline is T5-small fine-tuned identically but unconstrained; the standard model is pretrained T5-small, unfine-tuned. The monotonic model's initial loss is substantially higher (4.97 vs. 2.94), reflecting the sign-flipping softplus initialization of Section 3.

Despite this, the monotonic model converges reliably: validation loss decreases from 2.92 at epoch 1 to 2.54 at epoch 7, with rapid initial recovery (1.92 points in the first two epochs, from 4.97 to 3.05) then diminishing returns. A persistent 0.32-point optimization gap remains at convergence (2.54 vs. 2.21 for baseline), consistent with reduced expressivity—a deliberate trade-off, since structural regularity yields improved adversarial robustness (shown below).

Table 1: Training dynamics comparison (seed = 42).

Model	Initial Loss	Final Loss	Δ Loss	Epochs
Baseline	2.94	2.21	−0.73	6
Monotonic	4.97	2.54	−2.43	7

Table 2: Summarization quality on CNN/DailyMail (n = 200, seed = 42). Values are means with 95% bootstrap confidence intervals.

Model	ROUGE-1	ROUGE-2	ROUGE-L
Standard	32.6 [30.9, 34.2]	11.9 [10.5, 13.4]	26.6 [25.0, 28.1]
Baseline	30.9 [29.4, 32.4]	11.5 [10.1, 12.8]	25.0 [23.6, 26.4]
Monotonic	29.6 [28.0, 31.1]	10.6 [9.2, 11.9]	24.2 [22.7, 25.6]

Table 3: Summarization quality on XSUM (n = 200, seed = 42). Values are means with 95% bootstrap confidence intervals.

Model	ROUGE-1	ROUGE-2	ROUGE-L
Standard	19.9 [18.8, 21.1]	2.9 [2.3, 3.4]	13.3 [12.6, 14.1]
Baseline	20.0 [19.0, 21.0]	3.2 [2.7, 3.7]	13.3 [12.6, 13.9]
Monotonic	18.9 [18.0, 19.7]	2.8 [2.3, 3.2]	12.9 [12.3, 13.6]

Table 4: HotFlip attack results on CNN/DailyMail (n = 100, seed = 42).

Model	Avg. Deg.	Success Rate	ΔLoss
Standard	21.3%	69%	+0.42
Baseline	16.2%	63%	+0.35
Monotonic	5.2%	19%	+0.14

Table 2 reports summarization performance on CNN/DailyMail (n=200, seed 42): the monotonic model achieves ROUGE-L of 24.2 [22.7, 25.6] vs. 25.0 [23.6, 26.4] for the baseline (a 3.2 percent relative decrease), with a similar trend on ROUGE-1 (29.6 [28.0, 31.1] vs. 30.9 [29.4, 32.4], a 4.2 percent decrease), leaving the monotonic model competitive with substantial CI overlap across metrics.

Table 3 shows consistent results on XSUM (n=200, seed 42): ROUGE-L of 12.9 [12.3, 13.6] vs. 13.3 [12.6, 13.9] for baseline, a similar 3.0 percent gap—suggesting the cost of monotonicity is stable across CNN/DM and XSUM's distinct data distributions.

This recovery-then-plateau pattern continues: only 0.36 points of additional improvement occur from epochs 3 through 7, underscoring the extended warmup phase (15% vs. 10% for baseline) in stabilizing gradients under the constrained parameter space. Generation-length statistics (Appendix A.6) show no systematic length bias.

6.2 Adversarial Robustness

HotFlip Attacks. Following the attack and metric definitions of Section 4, Table 4 reports results on 100 test examples from seed 42; results were identical across two additional seeds (1337, 2024), confirming stability. Under HotFlip perturbations, the monotonic model incurs an average degradation of 5.2 percent, compared to 16.2 percent for the fine-tuned baseline and 21.3 percent for the standard pretrained model, with attack success rates of 19 percent, 63 percent, and 69 percent respectively—a 69.8 percent relative reduction in attack success rate and a 67.9 percent reduction in average degradation relative to the baseline (paired t-tests confirm high significance; Appendix A.6 gives exact statistics).

This large effect (Cohen's $d > 0.8$, $p < 10^{-9}$) arises without adversarial training or objective changes beyond the FFN constraints (Appendix A.6 restates the reduction).

Table 5: UAT trigger transfer matrix (seed = 42; $n = 100$). Entry (i, j) is the ROUGE-L delta (percentage points, attacked minus clean) when the trigger learned on model i is evaluated against model j. Diagonal entries (bold) are same-model attacks.

Trigger source \ Target	Standard	Baseline	Monotonic
Standard	**−0.40**	−0.45	−0.62
Baseline	−0.36	**+0.06**	+0.26
Monotonic	+0.42	+0.15	**−0.53**

Table 6: Constraint-scope screening on Pythia-1.4B (seed = 42). PPL denotes held-out Pile perplexity; SR denotes HotFlip attack success rate ($n = 200$, 10 flips). MLP-in was screened under the full training budget, the other scopes under a reduced budget.

Constraint scope	PPL base	PPL mono	SR base → mono	SR drop
MLP-in	6.67	24.32	71.5% → 83.5%	−12.0 pp
MLP-both	7.06	47.99	75.0% → 44.5%	+30.5 pp
MLP-in + Attn-out	7.06	88.17	75.0% → 35.5%	+39.5 pp

Universal Adversarial Trigger Attacks. Following the UAT protocol of Section 4 (80 training / 120 disjoint test examples), we measure NLL increase under trigger prepending. UAT attacks prove minimally effective across all models (NLL increases below 1 percent): the monotonic model shows the smallest degradation (0.73 percent) versus 0.89 percent for the baseline and 0.97 percent for standard T5, though effect sizes are too small for strong conclusions—consistent with monotonicity's benefits being most pronounced against adaptive, gradient-based rather than input-agnostic attacks.

6.3 Trigger Transferability

As with the HotFlip transfer check of Section 4.2, UATs give a first test of whether a success-rate gap is genuine or an artifact of each attack's own loss landscape: each trigger is learned independently per model, so evaluating trigger A against model B tests whether it captures a vulnerability specific to A or a shared weakness. Table 5 reports the full 3×3 transfer matrix (ROUGE-L delta on a held-out 100-example subset), each model's own trigger (rows) evaluated against every model (columns); seeds 1337 and 2024 match these entries to the reported precision.

Every entry lies within ±0.62 percentage points of ROUGE-L—an order of magnitude smaller than the HotFlip degradations above—with signs inconsistent within rows and columns and no row or column systematically larger or smaller than the others: no model is thus a systematically easier or harder UAT target, whether attacked by its own trigger or another's. Appendix A.6 gives the per-entry asymmetries and relates this pattern to the HotFlip transfer controls of Section 4.2.

6.4 Generalization to Decoder-Only Foundation Models

We next ask whether this robustness benefit is specific to the encoder-decoder setting or extends to decoder-only models, using the Pythia-1.4B track of Section 4.5.

Constraint scope determines whether robustness transfers. Table 6 reports the constraint-scope screening. The direct T5 analogue (MLP-in) fails on the decoder-only architecture: perplexity increases 3.65×, yet HotFlip success rate rises from 71.5% to 83.5%. Extending the scope closes this leak path: MLP-both reduces success rate by 30.5 points, and additionally constraining attention output (MLP-in + Attn-out) yields a 39.5-point reduction but roughly doubles the perplexity penalty (12.49× vs. 6.80×) for a further 9-point gain. We select MLP-both for full-budget validation; Appendix A.6 discusses the leak-path mechanism.

Table 7: Pythia-1.4B results with the MLP-both constraint scope under the full recovery budget. PPL denotes held-out Pile perplexity; SR denotes HotFlip attack success rate ($n = 200$, 10 flips); UAT NLL↑ denotes the relative NLL increase under the learned universal trigger (base → monotone).

Seed	PPL base	PPL mono	SR base → mono	UAT NLL↑
42	6.72	36.01	69.0% → 43.5%	+15.5% → +8.8%
1337	6.72	42.00	69.0% → 39.0%	+16.4% → +8.1%
2024	6.72	35.03	69.0% → 45.0%	+16.5% → +15.0%
Mean	6.72	37.68	69.0% → 42.5%	+16.1% → +10.6%

Multi-seed validation. Table 7 reports MLP-both under the full recovery budget across three seeds: HotFlip success rate falls stably from 69.0% to 39.0–45.0% (mean 42.5%)—a 26.5-point absolute, 38.4% relative reduction. Under UAT attacks, which unlike on T5 substantially damage the decoder-only baseline (NLL up ≈16%), monotone models are consistently less affected (mean NLL increase 10.6% vs. 16.1%), improving robustness against both attack types.

The quality–robustness trade-off is architecture-dependent. Robustness gains on Pythia-1.4B come at a markedly higher clean-performance cost than on T5: held-out perplexity increases by a mean factor of 5.61 (range 5.21–6.25), a gap full-budget recovery does not close, whereas T5's monotone model recovers to within a few percent of baseline (Section 7 explains this). The mechanism transfers across architectures, but scope and cost are architecture-conditional.

7 Discussion

This work provides empirical and theoretical support for monotonicity as a structural bias for language-model robustness: constraining feed-forward sublayers reduces adversarial vulnerability while largely preserving task performance. These gains trace to feed-forward sublayers—attention aggregates context while FFNs dominate nonlinear transformation—so constraining FFNs limits post-context refinement, aligning with classical monotone-systems theory, since monotonicity constrains how perturbations propagate through a model's internal state. The decoder-only experiments sharpen this picture: a decoder-only residual stream offers unconstrained routes around any single constrained matrix, so constraining only the feed-forward input fails on Pythia-1.4B while constraining both restores the benefit—monotonicity is a property of signal paths, not an isolated sublayer. In the encoder-decoder setting the trade-off is modest; in the decoder-only setting, broader coverage carries a substantially larger perplexity cost that full-budget recovery does not eliminate, since monotonicity constrains how representations change, not just their magnitude.

The main tables report T5-small and Pythia-1.4B; a scale-up protocol (Section 4.6) extends evaluation to larger variants once runs finish. Absent formal certificates, the HotFlip transfer controls (Section 4.2), order-preservation measurement (Section 4.3), and attribution ablations (Section 4.4) isolate whether the gap is genuine, whether order emerges through depth, and whether nonnegativity is causal. Consistent gains across architectures suggest a principled, scalable design.

8 Conclusion

We investigated monotonicity as a structural inductive bias for robustness: non-negativity constraints on feed-forward sublayers reduce adversarial vulnerability at modest quality cost in the encoder-decoder setting, transferring to decoder-only models—at larger perplexity cost—when both feed-forward projections are constrained. Theoretical guarantees, reducing this cost, and larger-scale evaluation remain future work.

Ethics statement

This work explores architectural constraints that improve the robustness of language models to adversarial manipulation. By studying monotonicity as a structural inductive bias, our approach contributes to the development of language models whose behavior is more predictable and analyzable, which is particularly relevant for safety-critical and decision-support applications. All experiments use publicly available pretrained checkpoints (T5, Pythia, Llama 3) and public benchmark datasets (CNN/DailyMail, XSUM, SAMSum, the Pile); the work does not involve human subjects or private data. The adversarial attacks studied (UAT, HotFlip) are standard robustness-evaluation techniques applied only to models under the authors' control, not a mechanism for attacking deployed third-party systems.

Reproducibility statement

Full training, evaluation, and attack configurations (learning rate, batch size, warmup schedule, seeds, decoding parameters) are given in Section 4. All reported confidence intervals use bootstrap resampling (1,000 resamples) and significance is assessed via paired t-tests with Bonferroni correction, as implemented in hpc_version/utils/stats_utils.py. Every reported run is tagged with its exact random seed and timestamp metadata.

AUTHOR: add the anonymized code/checkpoint release plan here (e.g., an anonymous repository link) before submission.

AI use statement

AUTHOR: review and finalize this statement before submission – confirm the scope below matches actual usage.

Generative AI tools (Claude, via Claude Code) were used to assist with software engineering tasks in this project, including developing and debugging the SLURM job-orchestration pipeline, experiment-tracking and results-aggregation scripts, and the automated test suite, and for editorial assistance in drafting and revising manuscript text. AI tools were not used to generate the theoretical claims, proofs, or experimental results reported in this paper; all reported numbers are produced by the training/evaluation/attack pipeline in hpc_version/ and foundation_llm_experiments/ and were verified by the authors against the underlying result JSON files before inclusion. All AI-assisted code and text were reviewed by the authors, who take full responsibility for the final content of this work, including any text, claims, or artifacts produced with the aid of generative AI.

References

Brandon Amos and J. Zico Kolter. Optnet: Differentiable optimization as a layer in neural networks. In International Conference on Machine Learning (ICML), 2017.

Cem Anil, Esin Durmus, Nina Panickssery, Mrinank Sharma, Joe Benton, Sandipan Kundu, Joshua Batson, Meg Tong, Jesse Mu, Daniel Ford, et al. Many-shot jailbreaking. Advances in Neural Information Processing Systems, 37:129696–129742, 2024.

Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In Proceedings of the 40th International Conference on Machine Learning, volume 202, pp. 2397–2430. PMLR, 2023.

Bochuan Cao, Yuanpu Cao, Lu Lin, and Jinghui Chen. Defending against alignment-breaking attacks via robustly aligned llm. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 10542–10560, 2024.

Hani Daniels and Marina Velikova. Monotone and partially monotone neural networks. IEEE Transactions on Neural Networks, 21(6):906–917, 2010.

Yue Deng, Wenxuan Zhang, Sinno Jialin Pan, and Lidong Bing. Multilingual jailbreak challenges in large language models. 2023. ArXiv, abs/2310.06474, 2024.

Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. HotFlip: White-box adversarial examples for text classification. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), pp. 31–36. Association for Computational Linguistics, 2018.

Hamid Reza Feyzmahdavian, Bart Besselink, and Mikael Johansson. Stability analysis of monotone systems via max-separable lyapunov functions. IEEE Transactions on Automatic Control, 63(3):643–656, 2017.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800GB dataset of diverse text for language modeling. arXiv preprint arXiv:2101.00027, 2020.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The Llama 3 herd of models. arXiv preprint arXiv:2407.21783, 2024.

Maya Gupta, Andrew Cotter, Jan Pfeifer, and Konstantin Voevodski. Monotonic calibrated interpolated look-up tables. In Advances in Neural Information Processing Systems (NeurIPS), 2016.

Morris W Hirsch. Systems of differential equations that are competitive or cooperative ii: Convergence almost everywhere. SIAM Journal on Mathematical Analysis, 16(3):423–439, 1985.

Nikolaus HR Howe, Ian R McKenzie, Oskar John Hollinsworth, Michał Zajac, Tom Tseng, Aaron David Tucker, Pierre-Luc Bacon, and Adam Gleave. Effects of scale on language model robustness. 2024.

Hiroshi Ito, Björn S Rüffer, and Anders Rantzer. Max-and sum-separable lyapunov functions for monotone systems and their level sets. In 53rd IEEE Conference on Decision and Control, pp. 2371–2377. IEEE, 2014.

Stasys Jukna et al. Boolean function complexity: advances and frontiers, volume 27. Springer, 2012.

David Khachaturov and Robert Mullins. Adversarial suffix filtering: a defense pipeline for llms. arXiv preprint arXiv:2505.09602, 2025.

Miles Q Li and Benjamin CM Fung. Security concerns for large language models: A survey. Journal of Information Security and Applications, 95:104284, 2025.

Anthony N Michel, Ling Hou, and Derong Liu. Stability of dynamical systems: On the role of monotonic and non-monotonic Lyapunov functions. Springer, 2015.

Laker Newhouse, R Preston Hess, Franz Cesista, Andrii Zahorodnii, Jeremy Bernstein, and Phillip Isola. Training transformers with enforced lipschitz constants. arXiv preprint arXiv:2507.13338, 2025.

An-Phi Nguyen, Dana Lea Moreno, Nicolas Le-Bel, and María Rodríguez Martínez. Mononet: enhancing interpretability in neural networks via monotonic features. Bioinformatics advances, 3(1):vbad016, 2023.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of machine learning research, 21(140):1–67, 2020.

Anders Rantzer and Bo Bernhardsson. Control of convex-monotone systems. In 53rd IEEE Conference on Decision and Control, pp. 2378–2383. IEEE, 2014.

Alexander Robey, Zifan Wang, and J. Zico Kolter. Smoothllm: Defending large language models against jailbreak attacks. arXiv preprint arXiv:2310.03684, 2023.

Davor Runje and Sharath M Shankaranarayana. Constrained monotonic neural networks. In International Conference on Machine Learning, pp. 29338–29353. PMLR, 2023.

Ahmed Ragab Mahmoud Salah. Jailbreaking llms: An in-depth security analysis of prompt injection vulnerabilities. 2026.

Davide Sartor, Alberto Sinigaglia, and Gian Antonio Susto. Advancing constrained monotonic neural networks: Achieving universal approximation beyond bounded activations. arXiv preprint arXiv:2505.02537, 2025.

Md Faiyaz Abdullah Sayeedi, Maaz Bin Hossain, Md Kamrul Hassan, Sabrina Afrin, Molla Md Sabit, and Md Shohrab Hossain. Jailbreaktracer: Explainable detection of jailbreaking prompts in llms using synthetic data generation. IEEE Access, 2025.

Joseph Sill. Monotonic networks. In Advances in Neural Information Processing Systems (NeurIPS), 1998.

Hal L Smith. Monotone dynamical systems: an introduction to the theory of competitive and cooperative systems. Number 41. American Mathematical Soc., 1995.

Jingtong Su, Julia Kempe, and Karen Ullrich. Mission impossible: A statistical perspective on jailbreaking llms. Advances in Neural Information Processing Systems, 37:38267–38306, 2024.

Eric Wallace, Shi Feng, Nikhil Kandpal, Matt Gardner, and Sameer Singh. Universal adversarial triggers for attacking and analyzing NLP. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 2153–2162. Association for Computational Linguistics, 2019.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, and Samuel R. Bowman. Adversarial glue: A multi-task benchmark for robustness evaluation of language models. In Advances in Neural Information Processing Systems (NeurIPS), 2021.

Haohua Wang, Jingge Wang, Zijie Zhao, Yang Tan, Yanru Wu, Hanbing Liu, Jingyun Yang, Enming Zhang, Xiangyu Chen, Zhengze Rong, et al. Understanding knowledge transferability for transfer learning: A survey. arXiv preprint arXiv:2507.03175, 2025a.

Yiwei Wang, Muhao Chen, Nanyun Peng, and Kai-Wei Chang. Vulnerability of large language models to output prefix jailbreaks: Impact of positions on safety. In Findings of the Association for Computational Linguistics: NAACL 2025, pp. 3939–3952, 2025b.

Maurice Weber, Randall Balestriero, and Richard Baraniuk. Certifying monotonic neural networks. arXiv preprint arXiv:2102.12566, 2021.

Siyu You, Yanzhi Ding, Marco Canini, Jan Pfeifer, and Maya Gupta. Deep lattice networks and partial monotonic functions. In Advances in Neural Information Processing Systems (NeurIPS), 2017.

Yifan Zhu, Yue Xie, Xin Chen, and Qiang Zhang. Autodan: Generating stealthy jailbreak prompts for large language models. In Advances in Neural Information Processing Systems (NeurIPS), 2023.

Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, and J. Zico Kolter. Universal and transferable adversarial attacks on aligned language models. arXiv preprint arXiv:2307.15043, 2023.

A Appendices

A.1 Mathematical Proofs

A.1.1 Proof of Lemma 3.3

Proof. For any $x, x' \in \mathbb{R}^n$ such that $x \leq x'$, monotonicity of f implies $f(x) \leq f(x')$. Applying monotonicity of g yields

$$g(f(x)) \leq g(f(x')),$$

which establishes the claim. □

A.1.2 Proof of Lemma 5.1

Proof. Recall $F(h) = h + N(h)$, so

$$J_F(h) = \nabla_h F(h) = I + J_N(h).$$

For $N(h) = W_2\sigma(W_1h + b_1) + b_2$ with $W_1, W_2 \geq 0$ elementwise,

$$J_N(h) = W_2 \operatorname{diag}(\sigma'(W_1h + b_1))W_1.$$

Since σ is elementwise non-decreasing, $\sigma'(z) \geq 0$ wherever it exists, so $\operatorname{diag}(\sigma'(\cdot)) \geq 0$ elementwise. Hence $J_N(h) \geq 0$ elementwise, and therefore $J_F(h) = I + J_N(h) \geq 0$ elementwise. □

Intuitively, increasing any hidden coordinate cannot decrease any output coordinate of F; sign cancellations internal to the FFN sublayer are ruled out entirely, and this holds unconditionally on the full d-dimensional hidden space, with no auxiliary projection matrix involved. Consequently, for any differentiable scalar objective $\mathcal{L} : \mathbb{R}^d \to \mathbb{R}$ defined on the sublayer's output (e.g., the loss induced by the rest of the network through $h' = F(h)$), the chain rule gives

$$\nabla_h \mathcal{L}(F(h)) = J_F(h)^\top \nabla_{h'} \mathcal{L}(h')\big|_{h'=F(h)}.$$

Thus the backpropagated gradient is filtered by $J_F(h)^\top$, whose entries are nonnegative under Lemma 5.1.

A.1.3 Proof of Lemma 5.3

Proof. By the factorization established above, $J_N(h_k) = W_2 \operatorname{diag}(\sigma'(W_1h_k + b_1))W_1$. Let S denote the (non-empty) subset of first-layer pre-activation indices with $(W_1h_k + b_1)_j \to +\infty$ as $k \to \infty$. By Assumption 5.2, $\sigma'((W_1h_k + b_1)_j) \to 0$ for $j \in S$, so the corresponding diagonal entries of $\operatorname{diag}(\sigma'(W_1h_k + b_1))$ vanish in the limit. Hence the contribution of the coordinates in S to the matrix product $W_2 \operatorname{diag}(\sigma'(W_1h_k + b_1))W_1$—i.e., to $J_N(h_k)$—vanishes as $k \to \infty$. Since $J_F(h_k) = I + J_N(h_k)$ and, by the chain rule, $\nabla_h \mathcal{L}(F(h_k)) = J_F(h_k)^\top \nabla_{h'} \mathcal{L}(h')|_{h'=F(h_k)}$, the contribution of the saturated coordinates in S to this gradient vanishes asymptotically as well. □

A.1.4 Extended Discussion: Saturation and ReLU

The saturation-based mechanism (Assumption 5.2, Lemmas 5.3–5.4) is stated for a general saturating activation because σ-style monotone MLPs in other settings (e.g., sigmoid- or softplus-gated units) exhibit it directly. Our T5-small experiments use ReLU FFN activations, for which $\sigma'(z) = 0$ exactly (not merely asymptotically) for all $z < 0$, but $\sigma'(z) = 1$ for all $z > 0$: ReLU saturates on the inactive (negative) side and never saturates on the active (positive) side. Assumption 5.2 and Lemmas 5.3–5.4 as stated describe saturation on the active side, which characterizes bounded activations such as sigmoid or tanh rather than ReLU. For ReLU, the non-negativity result of Lemma 5.1 still holds unconditionally, so the Jacobian-filtering mechanism applies directly to our models; a monotone increase in h can, however, move a previously inactive ReLU unit into its active region rather than deepen an existing saturation, so the persistence property of Lemma 5.4 does not transfer

to ReLU without modification. We therefore present the saturation-based mechanism as an additional attenuation effect relevant to monotone networks built from bounded activations, and rely on Lemma 5.1 alone as the mechanism directly applicable to the ReLU-based models studied here; characterizing an analogous persistence property for piecewise-linear activations is left to future work.

A.1.5 Proof of Lemma 5.4

Proof. Let $z_j(h) := (W_1 h + b_1)_j$. Since $W_1 \geq 0$ elementwise and $\delta \geq 0$,

$$z_j(h' + \delta) = (W_1(h' + \delta) + b_1)_j \geq (W_1 h' + b_1)_j = z_j(h').$$

Thus if the unit is saturated at h' (i.e., $\sigma'(z_j(h')) = 0$ in the saturated regime), it remains saturated at $h' + \delta$. □

A.2 Training and Evaluation Protocol

Both baseline and monotone models are fine-tuned from the same pretrained checkpoint under identical optimization settings. The main-text T5 tables use T5-small. Larger T5 arms (Section 4.6) keep the same optimizer and epoch budget and scale the effective batch size through gradient accumulation. We use the AdamW optimizer with learning rate 5×10^{-5} and weight decay 0.01, a per-device batch size of 4 on T5-small (reduced on larger tiers), and gradient clipping with norm 1.0. To accommodate the constrained parameterization, the monotone model employs an extended warmup phase of 15% of training steps, compared to 10% for the baseline. Both models are trained for 7 epochs and receive the same total training budget.

Training is conducted on the DialogSum, HighlightSum, and arXiv abstract datasets, comprising approximately 150K examples in total. Evaluation is performed on three held-out benchmarks: CNN/DailyMail (11,490 test examples), XSUM (11,334 test examples), and SAMSum (819 test examples). CNN/DailyMail is excluded entirely from training to assess out-of-distribution generalization.

At inference time, decoding is performed using beam search with 4 beams and a length penalty of 1.2, enforcing a minimum generation length of 10 tokens and a maximum length of 80 tokens. We additionally apply no-repeat n-gram blocking with $n = 3$ to discourage degenerate repetitions. All decoding parameters are fixed across models and evaluation sets. ROUGE-1, ROUGE-2, and ROUGE-L scores are computed using the rouge-score library with Porter stemming. We report bootstrap-based 95% confidence intervals using 1,000 resamples and assess statistical significance via paired t-tests with Bonferroni correction for multiple comparisons. Effect sizes are reported using Cohen's d.

A.3 Experimental Setup

The T5 tables in the main text report seed 42 on $n = 200$ evaluation subsets (CNN/DailyMail and XSUM). The validation protocol repeats the pipeline on five seeds (42, 1337, 2024, 8888, and 12345) with the full held-out test sets (CNN/DailyMail 11,490, XSUM 11,334, SAMSum 819), controlling randomness across Python, NumPy, and PyTorch. Cross-seed significance uses paired t-tests on seed-level means with Bonferroni correction across the metric family and paired Cohen's d; within each seed, per-example paired t-tests are computed on the ROUGE vectors of the baseline and monotone models. Completed five-seed and full-test numbers replace the subset tables when those JSON artifacts exist. Experiments run with deterministic algorithms where available, on NVIDIA A100 (40GB and 80GB), H200 (141GB), and RTX Pro 6000 (96GB) GPUs. The A100 environment uses PyTorch with CUDA 11.8; H200 and RTX Pro 6000 nodes use a CUDA 12.8 PyTorch build. Transformers 4.30 or later is used throughout.

A.4 Foundation Model Training Protocol

The decoder-only track fine-tunes Pythia-1.4B Biderman et al. (2023) on the uncopyrighted subset of the Pile Gao et al. (2020), with held-out evaluation texts drawn from a disjoint

streaming sample. Constraint-scope screening (Table 6) uses a reduced budget of 30,000 training samples at sequence length 1024 for 3–4 epochs on seed 42; full-budget validation of the selected MLP-both scope (Table 7) trains the baseline for 5 epochs and the monotone model for 10 epochs on each of three seeds (42, 1337, 2024). The extended monotone budget parallels the extended warmup used in the T5 track and compensates for the constrained parameterization's slower recovery. HotFlip evaluation uses 200 held-out examples with up to 10 token flips per example; UAT evaluation uses coordinate ascent with 3 restarts and 50 iterations, optimizing triggers on 80 texts and evaluating on 120 disjoint texts. Attack-time model evaluation is performed in bfloat16 precision. The Llama 3 scale-up route (Section 4.6) uses the same recovery budget, attack protocol, and order-preservation measurement, with the gated-updown constraint as the default scope. The ordered-pair probe protocol of Section 4.3 is applied to decoder residual streams with last-token pooling as the primary readout and mean pooling as a robustness check (the reverse of the pooling order used for the T5 encoder probes). Foundation-model experiments run on NVIDIA A100 (80GB) for Pythia-1.4B and on NVIDIA H200 (141GB) for the Pythia-2.8B/6.9B and Llama arms, under the SLURM workload manager, with per-stage checkpointing to permit resumption across scheduler preemptions.

A.5 Full Experimental Protocol

This section collects protocol details for the experiments introduced in Section 4 but not fully spelled out there for space.

HotFlip transfer and gradient-free controls (Section 4.2). The diagonal of the 2×2 (crafted-on $\times$ evaluated-on) transfer matrix reproduces the same-model HotFlip evaluation; the off-diagonal cells test whether substitutions found on one model remain damaging on the other. The random-substitution control applies the same number of flips as the corresponding HotFlip attack (five on the T5 track, ten on the Pythia track) at uniformly random positions, with replacements drawn from the same restricted candidate vocabulary used by the UAT attack, and involves no model or gradient in generating the substitutions. The query-based control performs a greedy per-position search that scores candidate flips by the target model's loss directly, with no gradients, so that a robustness gap that survives this control cannot be attributed to gradient masking of the HotFlip optimizer. The same protocol is used on both the T5 and Pythia tracks.

End-to-end order preservation (Section 4.3). The templated corpus strengthens along thirteen semantic axes (severity, safety constraint, certainty, urgency, obligation, and related directions) and yields 234 chains and 702 adjacent-level ordered pairs (x, x'), of which 195 (from a disjoint set of chains) are held out for evaluation so that probe directions are never fit on the pairs used to measure order. For each model we extract encoder hidden states at every layer (embeddings plus each of the six T5 encoder layers), pooled by mean over token positions, with final-token pooling as a robustness check. On the fit split (507 pairs) we train $p = 64$ logistic-regression probe directions $A \in \mathbb{R}^{p \times d}$ on the final-layer representations; these probes are a post-hoc measurement instrument and play no role in training or in the monotonicity constraint. On the evaluation split we report, per layer, the fraction of the p probe coordinates satisfying $Ah \leq Ah'$ for each ordered pair, with bootstrap 95% confidence intervals. A per-layer fraction near one means the network's semantic coordinates increase along the expected direction at that depth; a fraction that degrades with depth would indicate that unconstrained attention, LayerNorm, and residual mixing lose order even though each individual FFN sublayer is coordinatewise monotone.

Attribution ablation controls (Section 4.4). Parameter count is unchanged between arms, so a rank or parameter-count confound does not arise. In the sign-frozen control, $W = \mathrm{sign}(W_{\mathrm{pre}}) \cdot \mathrm{softplus}(V)$ reconstructs the pretrained mixed-sign pattern and preserves it throughout training via a frozen sign buffer, so the resulting FFN is not monotone. In the init-disruption control, $W = V$ is unconstrained but initialized at the same $|W_{\mathrm{pre}}| + \epsilon$ starting point as the monotone arm, isolating recovery training from a sign-disrupted initialization without any nonnegativity constraint. Both arms hold the training protocol, optimizer, and magnitude initialization fixed relative to the monotone run, and are screened on seed 42

through the same evaluation and attack stages as the main monotone run, extended to additional seeds if they move the reported metrics.

Scale-up protocol (Section 4.6). The main-text tables report the T5-small and Pythia-1.4B arms that are already complete. To test whether the quality–robustness pattern persists at larger capacity, the same pipelines accept a model-name override with size-tier presets (batch size, gradient accumulation, bfloat16, and gradient checkpointing). The T5 track is repeated on t5-base (220M) and t5-large (770M) for three seeds with full CNN/DailyMail, XSUM, and SAMSum test sets. The decoder-only track is repeated on Pythia-2.8B for three seeds (MLP-both) and screened on Pythia-6.9B for one seed before a three-seed extension.

A second decoder-only route uses better-known Llama 3 models: Llama-3.2-1B and Llama-3.2-3B for three seeds, with a one-seed screen on Llama-3.1-8B Grattafiori et al. (2024). These models replace the GPT-NeoX MLP with a SwiGLU feed-forward sublayer,

$$N(x) = W_{\text{down}}\big(\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x\big).$$

SiLU is not non-decreasing—its derivative is negative on $(-\infty, x^\star)$ for $x^\star \approx -1.28$—and the gate multiplies two input-dependent branches, so Proposition 3.4 does not apply as written. We therefore treat the Llama route as an empirical stress test of the nonnegativity constraint beyond the theory's assumptions, with the Stage 11 order-preservation measurement as the primary test of whether coordinatewise order still accumulates through depth. Two constraint variants are defined: gated-updown constrains W_{up} and W_{down} and leaves W_{gate} free, analogous to the partial scopes used on Pythia; gated-all constrains all three projections. Gated-updown is the default. The same gated-FFN code path also accepts Qwen2.5-1.5B and Qwen2.5-7B as a license-free fallback. These runs use the expanded CURC partitions: NVIDIA H200 (141GB) for the Pythia and Llama arms and NVIDIA RTX Pro 6000 (96GB) for the larger T5 arms, with A100 nodes retained for T5-small and Pythia-1.4B. Results from the scale-up arms replace the modest-scale limitation as they land; they are not substituted into the existing tables until the corresponding JSON artifacts exist.

A.6 Extended Results Discussion

This section collects secondary results-section discussion (Section 6) not fully spelled out there for space.

Generation length (Section 6, Training Dynamics and Summarization Quality). Both fine-tuned models produce longer outputs than the pretrained T5 model (mean lengths 74 to 76 tokens vs 57 tokens), reflecting adaptation to the distributional properties of the summarization training data. The monotonic model exhibits slightly higher length variance (std 11.3 tokens) compared to the standard model (std 12.0 tokens), though this difference is minimal. Importantly, the brevity penalty for the monotonic model (1.00) indicates that output length is well-calibrated to reference summary length, suggesting the constraints do not introduce systematic length bias.

HotFlip headline reduction (Section 6, Adversarial Robustness). Overall, enforcing monotonicity in the FFN sublayers yields a 69.8 percent relative reduction in attack success rate (from 63 percent to 19 percent) and a 67.8 percent reduction in average degradation (from 16.1 percent to 5.2 percent), without requiring adversarial training, modified training objectives, or architectural changes beyond the FFN constraints. Paired t-tests confirm the baseline-monotonic difference is highly significant (t=6.17, $p = 3.8\times10^{-9}$), with an even larger effect relative to the standard model (t=7.45, p=2.8×10^{-12}). The effect size is large (Cohen's $d > 0.8$) and the statistical significance is robust ($p < 10^{-9}$). These results demonstrate that structural constraints alone—applied selectively to feed-forward sublayers—can substantially improve robustness to gradient-based adversarial manipulation, providing evidence for monotonicity as a practical architectural inductive bias for building more robust language models.

UAT transfer-matrix asymmetries (Section 6.3). The monotonic model's own trigger is mildly more damaging to the standard and baseline models than to the monotonic model itself, while the standard model's trigger is comparably (in fact slightly more) damaging to the monotonic model than to the standard model. Under UATs, in other words, no model

is a systematically easier or harder target than any other, whether attacked by its own trigger or by another model's, which is the pattern expected if the near-zero UAT effect sizes documented above reflect UATs being a weak attack against sequence-to-sequence summarization models in general rather than a per-model vulnerability that happens to differ between the monotonic and non-monotonic models. Because UAT trigger search here is itself a gradient-free coordinate ascent over discrete tokens rather than a first-order attack, this transfer check complements, rather than substitutes for, the HotFlip substitution-transfer and gradient-free controls of Section 4.2, which probe the same distinction on the attack family that produces the larger and more differential effect sizes.

Pythia leak-path mechanism (Section 6.4, constraint-scope screening). The MLP-in failure mode indicates that adversarial gradient signal is routed around the constrained matrix through the unconstrained MLP output projection and attention pathways. The diminishing return from additionally constraining attention output (MLP-in + Attn-out), relative to MLP-both, is consistent with attention serving as a secondary leak path once the primary MLP-output path is closed.